

NAME: VELMURUGAN A

LAB EXPERIMENT : LESSION 3

EXPERIMENT: 1 AND 2

CHAPTER-3 CODES

[18]

```
def min_max(arr, low, high):  
    if low == high:  
        return arr[low], arr[low]  
    if high == low + 1:  
        if arr[low] < arr[high]:  
            return arr[low], arr[high]  
        else:  
            return arr[high], arr[low]  
    mid = (low + high) // 2  
    min1, max1 = min_max(arr, low, mid)  
    min2, max2 = min_max(arr, mid + 1, high)  
    return min(min1, min2), max(max1, max2)  
arr = list(map(int, input("Enter array: ").split()))  
minimum, maximum = min_max(arr, 0, len(arr)-1)  
print("Minimum =", minimum)  
print("Maximum =", maximum)
```

▼ ... Enter array: 5 7 3 4 9 12 6 2
Minimum = 2
Maximum = 12

[19]

✓ 4s

```
arr = list(map(int, input("Enter sorted array: ").split()))  
  
print("Minimum =", arr[0])  
print("Maximum =", arr[-1])
```

▼ ... Enter sorted array: 5 4 6
Minimum = 5
Maximum = 6

EXPERIMENT: 3

[20]

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr)//2
        left = arr[:mid]
        right = arr[mid:]
        merge_sort(left)
        merge_sort(right)
        i = j = k = 0
        while i < len(left) and j < len(right):
            if left[i] < right[j]:
                arr[k] = left[i]
                i += 1
            else:
                arr[k] = right[j]
                j += 1
            k += 1
        while i < len(left):
            arr[k] = left[i]
            i += 1
            k += 1
        while j < len(right):
            arr[k] = right[j]
            j += 1
            k += 1
    arr = list(map(int, input("Enter array: ").split()))
    merge_sort(arr)
    print("Sorted Array:", arr)
```



```
... Enter array: 31 23 35 27 11 21 15 28
Sorted Array: [11, 15, 21, 23, 27, 28, 31, 35]
```

EXPERIMENT: 4

[21]

```
count = 0
def merge_sort(arr):
    global count
    if len(arr) > 1:
        mid = len(arr)//2
        left = arr[:mid]
        right = arr[mid:]
        merge_sort(left)
        merge_sort(right)
        i = j = k = 0
        while i < len(left) and j < len(right):
            count += 1
            if left[i] < right[j]:
                arr[k] = left[i]
                i += 1
            else:
                arr[k] = right[j]
                j += 1
            k += 1
        while i < len(left):
            arr[k] = left[i]
            i += 1
            k += 1
        while j < len(right):
            arr[k] = right[j]
            j += 1
            k += 1
    list: arr
arr = list((6 items) [3, 34, 4, 12, 5, ...] t())
merge_sort(arr)
print("Sorted Array:", arr)
print("Comparisons:", count)
```

▼

```
... Enter array: 12 4 78 23 45 67 89 1
Sorted Array: [1, 4, 12, 23, 45, 67, 78, 89]
Comparisons: 16
```

EXPERIMENT: 5 AND 6

[22]

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[0]
    left = []
    right = []
    for x in arr[1:]:
        if x <= pivot:
            left.append(x)
        else:
            right.append(x)
    return quick_sort(left) + [pivot] + quick_sort(right)
arr = list(map(int, input("Enter array: ").split()))

print("Sorted Array:", quick_sort(arr))
```

▼

Enter array: 10 16 8 12 15 6 3 9 5
Sorted Array: [3, 5, 6, 8, 9, 10, 12, 15, 16]

[23]



```
def quick_sort(arr):

    if len(arr) <= 1:
        return arr

    pivot = arr[len(arr)//2]

    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]

    return quick_sort(left) + middle + quick_sort(right)

arr = list(map(int, input("Enter array: ").split()))
print("Sorted Array:", quick_sort(arr))
```

▼

... Enter array: 19 72 35 46 58 91 22 31
Sorted Array: [19, 22, 31, 35, 46, 58, 72, 91]

EXPERIMENT: 7

```
▶ arr = list(map(int, input("Enter sorted array: ").split()))
key = int(input("Enter key: "))
low = 0
high = len(arr) - 1
comparisons = 0
while low <= high:
    mid = (low + high) // 2
    comparisons += 1
    if arr[mid] == key:
        print("Element Found at Position:", mid + 1)
        print("Comparisons:", comparisons)
        break
    elif arr[mid] < key:
        low = mid + 1
    else:
        high = mid - 1
if low > high:
    print("Element Not Found")
```

```
*** Enter sorted array: 5 4 8
Enter key: 5
Element Not Found
```

EXPERIMENT: 8

```
[25] ▶ arr = list(map(int, input("Enter sorted array: ").split()))
key = int(input("Enter key: "))
low = 0
high = len(arr) - 1
while low <= high:
    mid = (low + high) // 2
    print("Low =", low,
          "High =", high,
          "Mid =", mid,
          "Value =", arr[mid])
    if arr[mid] == key:
        print("Element Found at Position:", mid + 1)
        break
    elif arr[mid] < key:
        low = mid + 1
    else:
        high = mid - 1
if low > high:
    print("Element Not Found")
```

```
▼ *** Enter sorted array: 3 9 14 19 25 31 42 47 53
Enter key: 31
Low = 0 High = 8 Mid = 4 Value = 25
Low = 5 High = 8 Mid = 6 Value = 42
Low = 5 High = 5 Mid = 5 Value = 31
Element Found at Position: 6
```

EXPERIMENT: 9 AND 10

[26]

✓ 8s

```
import math

points = [[1,3],[-2,2],[5,8],[0,1]]
k = int(input("Enter K: "))

points.sort(key=lambda p: math.sqrt(p[0]**2 + p[1]**2))

print("K Closest Points:", points[:k])
```

▼

```
Enter K: 2
K Closest Points: [[0, 1], [-2, 2]]
```

[27]

✓ 0s



```
from collections import defaultdict

A = [1,2]
B = [-2,-1]
C = [-1,2]
D = [0,2]

count = defaultdict(int)

for a in A:
    for b in B:
        count[a+b] += 1

answer = 0

for c in C:
    for d in D:
        answer += count[-(c+d)]

print("Number of Tuples:", answer)
```

▼

```
... Number of Tuples: 2
```

EXPERIMENT: 11 AND 12

[28]

✓ 13s

```
arr = list(map(int, input("Enter array: ").split()))
k = int(input("Enter k: "))

arr.sort()

print("K-th Smallest Element:", arr[k-1])
```

▼

```
Enter array: 12 3 5 7 19
Enter k: 2
K-th Smallest Element: 5
```

[29]

✓ 15s

```
def median_of_medians(arr, k):

    arr.sort()

    return arr[k-1]

arr = list(map(int, input("Enter array: ").split()))
k = int(input("Enter k: "))

print("Answer:", median_of_medians(arr, k))
```

▼

```
Enter array: 23 17 31 44 55 21 20 18 19 27
Enter k: 5
Answer: 21
```

EXPERIMENT: 13

[30]



```
from itertools import combinations
from bisect import bisect_left
arr = [45, 34, 4, 12, 5, 2]
target = 42
mid = len(arr) // 2
left = arr[:mid]
right = arr[mid:]
left_sum = []
right_sum = []
for r in range(len(left)+1):
    for subset in combinations(left, r):
        left_sum.append(sum(subset))
for r in range(len(right)+1):
    for subset in combinations(right, r):
        right_sum.append(sum(subset))
right_sum.sort()
best = None
min_diff = float('inf')
for s in left_sum:
    idx = bisect_left(right_sum, target - s)
    if idx < len(right_sum):
        total = s + right_sum[idx]
        if abs(target-total) < min_diff:
            min_diff = abs(target-total)
            best = int: target
    if idx > 0:
        total = 15 idx-1]
        if abs(target-total) < min_diff:
            min_diff = abs(target-total)
            best = total
print("Closest Sum =", best)
```



... Closest Sum = 41

EXPERIMENT: 14

[31]
✓ 0s

```
from itertools import combinations
from bisect import bisect_left

arr = [3, 34, 4, 12, 5, 2]
target = 15

mid = len(arr)//2

left = arr[:mid]
right = arr[mid:]

left_sum = []
right_sum = []

for r in range(len(left)+1):
    for subset in combinations(left, r):
        left_sum.append(sum(subset))

for r in range(len(right)+1):
    for subset in combinations(right, r):
        right_sum.append(sum(subset))

right_sum.sort()

found = False

for s in left_sum:
    remain = target - s

    idx = bisect_left(right_sum, remain)

    if idx < len(right_sum) and right_sum[idx] == remain:
        found = True
        break

print(found)
```

▼

True

EXPERIMENT: 15

[32]

✓ Os

```
A = [[1,7],  
      [3,5]]
```

```
B = [[1,3],  
      [7,5]]
```

```
a,b = A[0]  
c,d = A[1]
```

```
e,f = B[0]  
g,h = B[1]
```

```
P1 = a*(f-h)  
P2 = (a+b)*h  
P3 = (c+d)*e  
P4 = d*(g-e)  
P5 = (a+d)*(e+h)  
P6 = (b-d)*(g+h)  
P7 = (a-c)*(e+f)
```

```
C11 = P5 + P4 - P2 + P6  
C12 = P1 + P2  
C21 = P3 + P4  
C22 = P1 + P5 - P3 - P7
```

```
print("Result Matrix:")
```

```
print(C11, C12)  
print(C21, C22)
```

▼

```
Result Matrix:  
50 38  
38 34
```